

UML Use Case Diagram Guide

UML use case diagrams model **who** (actors) interacts with **what** (use cases) in a system. They consist of stick-figure actors, oval use cases, and a labeled system boundary box, connected by relationships. Use cases capture system functions (verb phrases); actors represent external roles. The diagram shows system scope and how each actor engages each use case ¹ ². Below are the core elements, relationships, and layout rules for creating correct UML use case diagrams.

Actors

- **Definition:** An *actor* is an external role (user, organization, or external system) that interacts with the subject system ³. Actors initiate or receive use-case behavior.
- **Notation:** Draw actors as stick figures labeled with a singular noun (e.g. `Customer`, `Admin`) ⁴ ². Names should use business-relevant terms, not implementation details ⁴.
- **Placement:** Place each actor *outside* the system boundary (the rectangle) and connect it by an association line to the use cases it participates in ⁴ ⁵. Conventionally, position primary (most important) actors toward the top-left of the diagram ⁶. Actors are roles, not specific individuals, and **do not** connect to other actors directly ⁴.
- **Inheritance (Generalization):** Actors can inherit from other actors (specialization). Draw a hollow-triangle generalization arrow from the specialized actor to the general actor. A specialized actor (child) automatically has all use cases of its parent role ⁷ ⁸. For example, a `RegisteredCustomer` actor can generalize (inherit from) `Customer`, meaning a registered customer has all the interactions of a customer plus potentially more.

Use Cases

- **Definition:** A *use case* is a scenario or service the system performs that provides value to an actor ¹. Each use case should deliver an observable result (a system function or goal). Use cases are named with goal-oriented verb phrases (e.g. *"Place Order"*, *"Validate Payment"*) ⁹. Use strong verbs and domain terms for clarity.
- **Notation:** Draw use cases as labeled ovals (horizontal ellipses) inside the system boundary ¹⁰ ¹. Each oval represents one distinct functionality. Label it with a concise description of the system action.
- **Placement:** Place major (primary) use cases toward the upper-left inside the boundary ⁹. You can stack use cases vertically to imply sequence or timing (top = first step, bottom = later steps) ¹¹. This is a visual cue (not formal UML) but commonly used for readability.
- **Generalization:** Use generalization (inheritance) for use cases when one is a more specific version of another ("is-a" relationship). Draw a hollow-triangle arrow from the specialized use case up to the general use case ¹² ¹³. The child use case inherits the flow of the parent. For example, "Pay by Credit Card" and "Pay by Bank Transfer" can both generalize a more general "Pay Bill" use case ¹⁴. Apply this only when one scenario truly specializes another (the "is-like" rule) ⁸.

System Boundary and Scope

- **System Boundary:** Draw a rectangle around all use cases to define the system's boundary or scope ¹⁵ ¹⁶ . Label this box with the system (subject) name (e.g. *"Online Store"*). Everything inside is part of the system's functionality; actors remain outside. The boundary box makes clear which functions belong to the system vs. external entities ¹⁵ ¹⁶ .
- **Scope and Releases:** Use cases inside this box are to be implemented; outside elements are external. You can use multiple labeled boxes (subsystems or releases) to partition complex systems, but avoid meaningless extra boxes ¹⁷ . Each box's label should reflect the system or release name to clarify scope.

Relationships

Association

An **association** is a solid line connecting an actor to a use case ¹⁸ . It means the actor participates in that use case's interaction. (Associations can also link two use cases, but in use-case diagrams they usually connect actors to use cases.) Typically, no arrowhead is drawn on actor-use-case associations (the arrow is optional in UML but often omitted for simplicity) ⁵ . An actor can have many associations (to many use cases), and a use case can connect to multiple actors ¹⁹ .

- Use an association whenever the actor performs or triggers the use case ⁵ . For example, connect **Customer** to **TransferFunds** if the customer initiates that use case.
- Draw the line straight with minimal crossings. (Avoid connecting actors directly to other actors.)
- Label the line only if needed (usually associations are unlabeled).

«include»

The **«include» relationship** (dashed arrow labeled «include») goes from a **base** use case to an **included** use case ²⁰ ²¹ . It means the base use case *always* invokes the included use case as part of its flow. This is used to factor out and reuse common behavior. For instance, if many transactions require authentication, you might have multiple use cases (e.g. *"Pay Bill"*, *"Transfer Funds"*) each «include» a *"Validate User"* use case. In an example, the **Compose Post** use case includes **Add Image**: every time a post is composed it *includes* adding an image ²² ²³ . The key rules are:

- The included use case is mandatory – the base use case is incomplete without it ²³ . The base use case typically calls the include use case at a known point ²⁴ .
- Use «include» only between use cases (never between an actor and a use case).
- Convention: draw the included use case to the right of the base use case, with the dashed arrow pointing toward the base ²⁵ . Label the arrow «<<include>>».

«extend»

The **«extend» relationship** (dashed arrow labeled «extend») goes from an **extension** use case to a **base** use case ²⁶ ²⁷ . It indicates optional or conditional behavior that augments the base case. The base use case remains meaningful on its own; the extension adds extra steps under certain conditions. For example, a **BookFlight** use case may have an extending **SelectSeat** use case: seat selection is **optional**, so

SelectSeat «extends» BookFlight when the passenger chooses to pick a seat ²⁸. In another example, an order process could have an extend SpecifyShipping use case that only runs if the user opts to provide extra shipping instructions ²⁹. Key points:

- The extension use case *depends* on the base: the base runs by itself, and the extension adds behavior only when triggered ³⁰ ³¹.
- Use «extend» for optional alternate flows, not for shared behavior (that's for «include»).
- Convention: draw the extending use case below the base use case, with a dashed arrow from the extension up to the base ³². Label it «<<extend>>».

Generalization (Inheritance)

A **generalization** is an “is-a” inheritance arrow (a solid line with a hollow triangle). It is used for both actors and use cases ¹² ³³. The arrow points from the *specialized* element up to the *more general* element. A child element inherits all of the parent's behavior:

- **Actor generalization:** A specialized actor (child) automatically has all the use cases of its parent. For example, you might have an actor Employee and a specialized actor Manager; draw a generalization arrow from Manager to Employee to show that a manager can do everything an employee does (and possibly more) ⁷ ⁸.
- **Use-case generalization:** Similarly, a specialized use case inherits the flow of the general use case. For example, RentCar and RentBike can generalize RentVehicle (both are kinds of renting vehicles) ³⁴. Each child use case is “like” the parent plus extra specifics (the “is-like” rule) ⁸ ¹⁴.

In diagrams, place child elements below their parent and connect with the generalization arrow ⁸. Label is not needed beyond the shape (sometimes the stereotype «{actor}» is used on actors, but typically just name the actors/use-cases plainly).

Layout and Presentation Guidelines

- **Diagram Boundary:** Only include actors and use cases that belong to this system. All use cases go inside the system box; all actors go outside. Draw a clear rectangular boundary and label it with the system name ¹⁵ ¹⁶.
- **Actor Placement:** Actors are drawn outside. Put the primary actor(s) (most important role) toward the top-left ⁶. Other actors can be placed around as needed. Never put actors inside the boundary.
- **Use Case Placement:** Keep use cases inside. Place key use cases (main features) in the upper-left inside the box ⁹. If depicting sequence, stack related use cases vertically (early steps above later steps) ¹¹.
- **Include/Extend Layout:** By convention, draw included use cases to the **right** of their base use cases, and extending use cases **below** their bases ³⁵. This spatial separation helps readers distinguish the relationships visually.
- **Generalization Layout:** Place specialized elements below the general ones. Draw generalization arrows pointing upward (triangle touching the parent) ⁸. Align these arrows vertically if possible to avoid crossing other lines.
- **Naming and Stereotypes:** Label use cases clearly (strong verbs, e.g. “Checkout Cart”). Label relationships explicitly with «include» or «extend». For any actor representing the system itself or another internal system, use the stereotype «system» (e.g. <<system>> AutomatedAgent) ³⁶.

- **Clarity:** Do not cross lines when possible, and avoid unnecessary detail. Show only relevant actors and use cases to keep the diagram readable ¹⁵ ⁴ . Uniformly use stick figures for actors and ovals for use cases ¹⁰ ² . Ensure all names are consistent and meaningful.

By following these UML conventions, your use case diagram will unambiguously show each actor's roles and how use cases relate via <<include>>, <<extend>>, and generalization. Citations to UML standards and guidelines are shown above ¹ ³⁷ ³⁸ to ensure correctness.

Sources: Authoritative UML documentation and UML modeling guides ¹ ²⁶ ³⁹ ²⁷ ³⁷ were used to compile these rules and examples. Each element and relationship is defined per UML semantics, and layout tips are drawn from standard practice ¹⁰ ⁴⁰ ⁸ .

¹ ³ ¹² ¹⁵ ¹⁸ ²⁰ ²² ²⁶ ²⁸ ³⁴ Use Case Diagram - Unified Modeling Language (UML) - GeeksforGeeks
<https://www.geeksforgeeks.org/system-design/use-case-diagram/>

² ⁴ ⁵ ⁶ ⁸ ⁹ ¹⁰ ¹¹ ¹⁶ ¹⁷ ²⁴ ²⁵ ³² ³⁵ ³⁶ ³⁸ ³⁹ ⁴⁰ UML Use Case Diagrams: Diagramming Guidelines
<https://agilemodeling.com/style/usecasediagram.htm>

⁷ ¹³ ¹⁴ ¹⁹ ²³ ³¹ Use Case Diagram Relationships Explained with Examples - Creately Blog
<https://creately.com/blog/diagrams/use-case-diagram-relationships/>

²¹ ³³ ³⁷ UML relationships in use-case diagrams
<https://www.ibm.com/docs/en/dma?topic=diagrams-relationships-in-use-case>

²⁷ ²⁹ ³⁰ Extend relationships in use-case diagrams
<https://www.ibm.com/docs/en/dma?topic=diagrams-extend-relationships>